LogInstruct: Knowledge-Driven Instruction Synthesis for Enhancing LLM-Based Log Analysis

Lipeng Ma, Yixuan Li, Weidong Yang, *Member, IEEE*, Mingjie Zhou, Mingyu Zhao, Ben Fei, *Member, IEEE*, Shuhao Li, *Graduate Student Member, IEEE*, Sihang Jiang, and Yanghua Xiao

***Abstract*—Logs are vital for monitoring service health and troubleshooting in large-scale online service systems. Recent advancements in large language models (LLMs) have demonstrated potential for automated log analysis. However, existing general LLMs face two key challenges in log analysis: a lack of domain-specific knowledge and limited adaptability to diverse log analysis tasks. To address these limitations, we propose LogInstruct, a novel LLM-based knowledge-driven instruction synthesis framework designed to enhance the log analysis capabilities of LLMs. LogInstruct bridges the domain knowledge gap by organizing log-specific knowledge into a structured knowledge tree to guide the synthesis of instruction. To ensure the diversity of instructions and hierarchical log analysis skills, we leverage Bloom's Taxonomy to synthesize log instructions of varying difficulty levels, not only covering a wide range of log analysis scenarios but also progressively challenging the model's reasoning abilities. LogInstruct synthesizes 88 K high-quality instruction datasets without manual intervention. Experiments demonstrate that open-source LLMs fine-tuned on our dataset achieve state-of-the-art performance across five distinct log analysis tasks, outperforming proprietary LLMs such as GPT-4o.**

***Index Terms*—Log analysis, large language model, data synthesis, instruction tuning.**

I. Introduction

LOGS serve as critical records of service runtime status, such as cloud platforms and edge clouds, playing a pivotal role in troubleshooting and monitoring service health [1], [2], [3]. Traditional log analysis methods, which rely heavily on experienced specialists manually inspecting vast amounts of log data, are often labor-intensive and inefficient [4]. Thus, automated log analysis has emerged as a promising solution, leveraging algorithms and machine learning models to extract meaningful insights from log data [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. In recent years, the remarkable success of large language models (LLMs) in natural language processing (NLP) has spurred growing interest in their application to automated log analysis. By integrating advanced prompting strategies and capitalizing on their powerful understanding and reasoning capabilities, LLM-based approaches [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25] have demonstrated significant progress in log analysis tasks such as log parsing and anomaly detection.

Despite these advancements, LLMs still face two critical limitations when applied to log analysis. First, they suffer from a domain knowledge gap. Logs are typically concise, highly condensed, and filled with domain terminology [17], [20], [26], [27], [28], such as *IP, Port, Parameters*, yet general LLMs are trained on general-purpose datasets, limiting their ability to understand logs effectively. This knowledge gap results in inaccurate inferences. Second, LLMs exhibit limited adaptability to diverse log analysis tasks. Real-world log analysis tasks, from simple log interpretation to complex fault diagnosis and solution recommendation, require distinctly different analysis logic and thought. While general LLMs are optimized for general-purpose human intents [29], [30], they lack specialized reasoning capabilities and do not align with the task-specific demands of log analysis.

To adapt general-purpose LLMs to specific domains, supervised fine-tuning (SFT) [31], [32], [33] on high-quality instruction datasets has proven effective across specialized domains such as code generation [34], [35], medical applications [36], and legal analysis [37]. However, constructing high-quality instruction datasets for log analysis presents unique challenges. On one hand, manually creating high-quality instruction data is labor-intensive and costly. On the other hand, existing public datasets for log analysis are limited in diversity, failing to generalize across real-world scenarios. For example, the previous study [12] has shown that the log duplication rate in the public datasets HDFS and Thunderbird exceeds 99% . Additionally, these public datasets frequently exhibit simplistic data patterns [12], [38], limiting their ability to develop advanced log analysis capabilities. In real-world scenarios, high-quality log analysis data currently remains dependent on expert-labeled data.

To overcome the challenges of manual data construction, LLM-based data synthesis has recently emerged as a promising alternative [39], [40]. However, applying these methods to log analysis introduces two critical challenges. First, general-purpose LLMs often generate inaccurate or irrelevant

Received 6 August 2025; revised 17 December 2025; accepted 7 January 2026. Date of publication 21 January 2026; date of current version 10 April 2026. This work was supported in part by the Natural Science Foundation of Shanghai under Grant 24ZR1405000 and in part by the Postdoctoral Fellowship Program of CPSF under Grant GZC20242252. *(Corresponding authors: Weidong Yang.)*

Lipeng Ma, Yixuan Li, Weidong Yang, Mingjie Zhou, Shuhao Li, Sihang Jiang, and Yanghua Xiao are with the School of Computer Science, Fudan University, Shanghai 200433, China (e-mail: lpma21@m.fudan.edu.cn; yxli24@m.fudan.edu.cn; wdyang@fudan.edu.cn; mjzhou19@fudan.edu.cn; shli23@m.fudan.edu.cn; jiangsihang@fudan.edu.cn; shawyh@fudan.edu.cn).

Mingyu Zhao is with the School of Artificial Intelligence, Optics and Electronics (iOPEN), Northwestern Polytechnical University, Xi'an 710072, China (e-mail: myzhao@nwpu.edu.cn).

Ben Fei is with the Department of Information Engineering, the Chinese University of Hong Kong, Hong Kong (e-mail: benfei@cuhk.edu.hk).

Our source code and synthesized data are available at https://github.com/LeaperOvO/LogInstruct.

Digital Object Identifier 10.1109/TSC.2026.3656204

log analysis instructions. The task of log analysis is knowledge-intensive, requiring a comprehensive knowledge chain that encompasses familiarity with the syntax and semantics of logs, recognition of patterns and anomalies, and ultimately, an understanding of troubleshooting strategies to resolve faults. This knowledge gap makes it challenging to synthesize effective log analysis instructions. Second, covering a wide range of log analysis tasks with varying cognitive difficulty is essential but challenging. Real-world log analysis involves different levels of analytical difficulty, from simple log interpretation to complex fault resolution. Without explicit modeling of difficulty hierarchy, LLMs may develop fragmented log analysis competencies, excelling at simple pattern matching but collapsing when faced with complex troubleshooting scenarios.

In this work, we propose LogInstruct, a novel LLM-based knowledge-driven instruction synthesis framework designed to enhance the LLMs' log analysis capabilities. Specifically, to bridge the knowledge gap, we first introduce domain knowledge for each log from the documentation, such as background descriptions and solution strategies, and organize this fragmented knowledge into a structured knowledge tree that presents knowledge clearly and systematically, facilitating LLMs to fully utilize this information for synthesis instructions. To ensure the diversity of instructions and hierarchical log analysis skills, we introduce Bloom's Taxonomy [41], [42], a hierarchical cognitive framework that helps synthesize log instructions of varying difficulty levels based on the knowledge tree. By mapping log analysis tasks onto different levels of cognitive difficulty, synthesized instructions that not only cover a wide range of log analysis scenarios but also progressively challenge LLM's reasoning abilities.

Using LogInstruct, we synthesize 88K high-quality instructions for log analysis without manual intervention. We evaluate our framework by fine-tuning Qwen2.5 of varying sizes on the synthesized instructions and assessing its performance across five distinct log analysis tasks. Our experiments demonstrate that models fine-tuned on our synthesized data achieve state-of-the-art (SOTA) performance, significantly outperforming proprietary LLMs such as GPT-4o. Additionally, our findings reveal critical insights into LLM instruction fine-tuning for log analysis: domain knowledge accumulation is critical, and instruction hierarchy matters. These findings provide actionable guidelines for future research in LLM-driven log analysis.

This paper presents the following key contributions:

1) To our best knowledge, we are the first to introduce the knowledge-driven instruction synthesis framework designed for log analysis. By leveraging LLMs as data synthesis engines and integrating domain knowledge, our framework synthesizes 88K high-quality instructions without manual intervention.
2) We introduce the domain knowledge of logs for ensuring the accuracy and relevance during instruction synthesis. To ensure diversity, we introduce hierarchical instruction synthesis based on Bloom's Taxonomy, facilitating the development of diverse log analysis capabilities.
3) We synthesize 88K instructions and demonstrate their effectiveness through fine-tuning and extensive experiments, achieving SOTA performance and providing critical insights for future research.

II. Related Works

A. Large Language Models for Log Analysis

Recent advances in LLMs have demonstrated significant potential in automated log analysis tasks, addressing critical tasks ranging from foundational log parsing to advanced failure diagnostics. For instance, LLMParser [43] systematically evaluates various LLMs on log parsing and demonstrates that LLMs significantly outperform state-of-the-art parsers by leveraging their in-context learning capabilities. Similarly, frameworks like LogPrompt [44], DivLog [15], LILAC [18], SelfLog [45], and LogBatcher [46] showcase the effectiveness of LLMs in parsing structured and semi-structured logs without the need for extensive task-specific fine-tuning. Beyond log parsing, LLMs have also been applied to higher-order analysis tasks. Methods like LogGPT [47], LogPrompt [44], LogLLM [48], LogExpert [21], AdaptiveLog [22], and OpenRCA [24] utilize LLMs for anomaly detection, root cause analysis, and solution recommendation.

Despite their promise, domain knowledge gaps and limited adaptability remain challenges when adapting LLMs for log analysis. To overcome these challenges, we introduce a novel knowledge-driven instruction synthesis framework, enabling LLMs to improve their log analysis capabilities with instruction tuning.

B. Synthesizing Instruction Tuning Data

Due to the huge computational overhead of pre-training, instruction tuning has become essential for adapting LLMs to domain-specific tasks [31], [32], [33]. By training on high-quality instruction datasets, researchers have successfully enhanced their performance in areas such as code generation [34], [35], medical diagnosis [36], and legal analysis [37]. However, constructing high-quality datasets for domain-specific tasks like log analysis is particularly challenging due to the diversity of scenarios and the cost of annotation. To address the high costs of manual annotation, researchers have turned to LLM-based data synthesis, which involves generating synthetic instruction datasets using LLMs [39], [40]. However, general-purpose LLMs often suffer from hallucination and inaccuracy when applied to the highly technical field of log analysis. Furthermore, ensuring diversity and relevance in synthesized instructions remains a critical challenge. Existing general-purpose instruction synthesis works [32], [49], [50] focus on human instruction alignment and cannot be applied to industrial log analysis scenarios.

Considering the LLM-based data synthesis technique for log analysis remains underexplored, to address these limitations, we explore leveraging LLM to synthesize instruction data tailored specifically for log analysis. By constructing a structured knowledge tree to integrate domain-specific information and

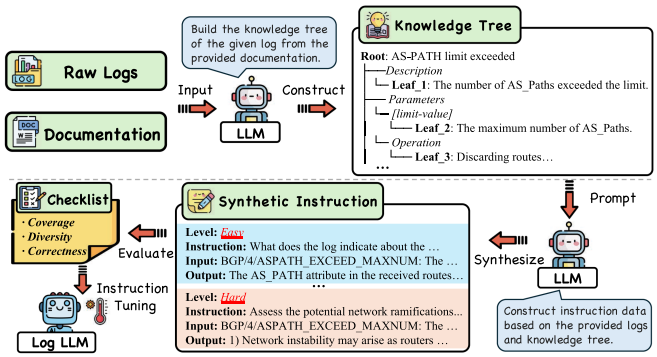

Fig. 1. The pipeline of LogInstruct.

leveraging Bloom's Taxonomy to model instructions of varying difficulty, LogInstruct ensures both accuracy and diversity in the generated data.

III. Methods

A. Overview

Fig. 1 shows the overall pipeline of LogInstruct, which is divided into two stages: knowledge tree construction and instruction synthesis. In the knowledge tree construction stage, we collect raw logs and corresponding documentation as input, which provides sufficient background information, such as description, cause, and solutions. Then we employ the LLM to extract knowledge and organize it into a structured knowledge tree. In the instruction synthesis stage, we take the constructed knowledge tree as the foundation, and then employ the LLM to synthesize instructions of different difficulty levels. After instruction synthesis, we also evaluate and refine the synthesized instructions to ensure the quality of the instructions. Finally, these synthesized instructions will be used to fine-tune the open-source LLM and get the log analysis-specific LLM. In the following sections, we describe the details of LogInstruct.

B. Knowledge Tree Construction

LLMs are usually unable to understand logs accurately when they lack domain knowledge, which leads to the inability to generate effective log analysis instructions. To mitigate this, we incorporate domain-specific knowledge for each log entry by leveraging publicly available documentation. Documentation serves as a repository of domain-specific knowledge, offering detailed background information, such as descriptions of log events, their causes, and potential solutions.

However, documentation is often verbose and fragmented, which poses challenges for LLMs to effectively extract and utilize key knowledge. To address this, we systematically organize the extracted information into a knowledge tree structure for each log entry. This structured organization enables more precise knowledge extraction and facilitates the generation of high-quality log analysis instructions.

We construct a knowledge tree $Tree_i$ for each log log_i, the knowledge tree $Tree_i = root_i \bigcup \{node_i^k\}_{k=1}^m \bigcup \{leaf_i^k\}_{k=1}^n$ comprises three fundamental components: 1) *Root Node* $root_i$, which represents the core issue derived from the log message, synthesized into a concise phrase (e.g., "Database Connection Failure"). It serves as the semantic anchor for subsequent knowledge organization. 2) *Branch Nodes* $\{node_i^1, node_i^2, ..., node_i^m\}$, these nodes serve as intermediate categories that organize domain knowledge into different groups (e.g., error causes, parameter information, impact scope). The identification and granularity of these branches are determined automatically by the LLM, guided by prompts to classify and, when appropriate, further subdivide knowledge categories based on logical coherence. 3) *Leaf Nodes* $\{leaf_i^1, leaf_i^2, ..., leaf_i^n\}$, which represent the most atomic knowledge units, each encapsulating a complete and non-redundant fact derived from the documentation. This hierarchical organization guarantees that each root-to-leaf path forms a complete, non-redundant knowledge chain, ensuring both comprehensive coverage and logical clarity.

The domain knowledge for each log entry is sourced from its corresponding publicly available documentation, which is typically in HTML format. We first employ XPath queries to precisely extract the relevant textual information for each log. To transform these unstructured text passages into a form usable for instruction synthesis, we leverage the semantic understanding capability of LLMs with prompt engineering. This process converts raw text into a hierarchical Knowledge Tree. The construction proceeds through three distinct stages:

1) *Root Node Identification:* The LLM analyzes the raw log message to identify its core semantic issue (e.g., "Database Connection Failure"), ignoring variable parameters. This becomes the root node of the tree.
2) *Branch Node Categorization:* The LLM classifies related knowledge from documentation into distinct categories and summarizes the category with keywords as the branch node. To ensure a consistent starting point for categorization across different logs, the prompt supplies a set of seed categories (e.g., "Description", "Causes", "Impact"). The LLM is instructed to classify the documentation content into these or other logically distinct categories, using a keyword or short phrase to label each branch node. For each branch, the LLM is prompted to check if the category contains subcategories or heterogeneous information. If so, it refines the branch to ensure logical clarity and capture multi-level details. For example, the possible causes of the log can be further divided into direct causes and indirect causes.
3) *Leaf Node extraction:* Finally, for the content under each terminal branch, the LLM is prompted to extract atomic, non-redundant factual statements. Verbose descriptions are condensed into concise knowledge units, which become the leaf nodes attached to their corresponding branches. To ensure knowledge richness, we filter out knowledge trees with fewer than three leaf nodes.

To ensure seamless integration with downstream stages of our framework, we constrain the LLM to output the final Knowledge Tree in a rigorously defined JSON schema. This schema

mandates the structure, node types, parent-child relationships, and text formatting. To promote consistency and handle noisy documentation, we implement three key mechanisms. First, any output that violates the predefined JSON schema will be automatically captured, triggering re-generation attempts. If violated again, it will be directly discarded. Second, after tree construction, the structured JSON is fed back to the LLM with a prompt asking it to verify the logical coherence of each root-to-leaf path and to identify any contradictory information or logical gaps. Based on this feedback, specific branches can be revised. Third, for missing information, we instruct the LLM not to fabricate content. If documentation lacks critical details, omit the corresponding branches or leaf nodes rather than filling in with assumptions. For redundant information, the prompt for leaf node generation explicitly requires "non-redundant" fact extraction, guiding the LLM to merge semantically equivalent statements.

C. Instruction Synthesis

The instruction synthesis module (as shown in Fig. 2) is responsible for creating high-quality instruction data tailored to log analysis tasks, which comprises two components: *LLM-driven Instruction Generation* and *Evaluation and Refinement*. Details of both components are described next.

1) Instruction Generation: The process of instruction generation is focused on constructing synthetic data with the LLM that facilitates fine-tuning of open-source LLM for log analysis tasks. The generated data is structured into three fields: *"instruction"*, *"input"*, and *"output"*. The instruction field represents the query to the model, the input field contains raw log data, and the output field provides the expected reasoning or response. To avoid ambiguity, we denote the *"instruction"* field by instruction question and the *"output"* field by response. The instruction generation process takes as input a raw log entry and its corresponding knowledge tree, and the LLM is required to traverse the knowledge tree based on breadth-first traversal and cover more than 90% of the leaf nodes. The generation process comprises two main steps: Question Generation and Response Generation.

Question Generation: To comprehensively design a variety of real-world log analysis questions and capture different analytical perspectives, we draw inspiration from Bloom's Taxonomy [41], [42]. Specifically, we utilize prompting strategies to systematically guide the LLM in generating diverse log analysis questions that span a range of cognitive difficulty levels, grounded in the structured knowledge tree. By explicitly defining difficulty requirements within the prompts, the LLM can craft different questions that range from straightforward factual queries to advanced multi-step reasoning tasks. This approach ensures that the model is trained on instructions across multiple cognitive levels, enhancing its ability to adapt to a wide array of log analysis scenarios. Specifically, we design three different instruction difficulties:

- *L1 (Easy - Remember & Understand):* L1 questions are designed around a single path within the knowledge tree and focus on fundamental knowledge and comprehension. These instructions provide a basic understanding of log content, requiring the LLM to identify core issues or facts without the need for advanced reasoning. This helps the model acquire essential domain knowledge and bridge existing gaps in general LLMs.
- *L2 (Medium - Apply & Analyze):* L2 questions emphasize localized analysis and reasoning that span multiple branches of the knowledge tree. These instructions require the LLM to integrate and apply knowledge from several related information, fostering a deeper understanding of the log context and encouraging more advanced analytical thinking.
- *L3 (Hard - Synthesize & Predict):* L3 questions focus on comprehensive analysis across the entire knowledge tree and require the LLM to conduct system-level reasoning, often involving hypothetical scenarios or multi-step problem-solving. The LLM design questions from a holistic perspective, considering interactions across the full scope of the knowledge tree, to address more complex and realistic log analysis tasks.

To explicitly model the knowledge sources of each instruction question, the LLM plans how to utilize different paths within the knowledge tree to design questions of varying difficulty. This ensures that the generated instructions are logically grounded in domain knowledge. To maintain balance across difficulty levels, we generate three instructions for L1 and 2–3 instructions each for L2 and L3 per log entry. Additionally, we ensure that the generated queries are diverse in phrasing and structure, avoiding repetitive instruction questions.

Response Generation: The responses are synthesized by leveraging the knowledge tree to provide accurate and contextually relevant answers to the questions. The complexity of the response depends on the difficulty level of the instruction. Specifically, for L1 instructions, responses are concise and directly summarize factual information from the knowledge tree. For L2 instructions, responses provide step-by-step reasoning, integrating knowledge from multiple branches of the knowledge tree. For L3 instructions, responses involve predictive reasoning and systematic analysis, describing the potential impacts or outcomes of specific scenarios. Responses for L2 and L3 instructions emphasize clarity and logical coherence, avoiding repetition of knowledge tree content while maintaining alignment with the domain knowledge.

The final instruction data is output in a structured JSON format for convenient data acquisition, including a detailed plan for instruction generation and the synthesized data.

2) Evaluation and Refinement: To ensure the quality and diversity of the generated instructions, a robust evaluation and refinement process is implemented in the LogInstruct framework. This process is designed to systematically evaluate the generated instructions and refine them to meet predefined quality standards. Specifically, we evaluate the quality of the generated instructions along three key dimensions: **Coverage**, **Diversity**, and **Correctness**. Instructions failing to meet the requirements in any of these dimensions are refined with the LLM, ensuring the final instruction dataset is comprehensive, diverse, and accurate.

1) Coverage: The coverage dimension measures the comprehensiveness of the instructions in terms of the tree paths, ensuring that the synthesized instructions fully utilize the tree paths to generate questions and answers that cover all relevant aspects of the log. For evaluation, we compute the ratio of the number of utilized leaf nodes to the total number of leaf nodes in the knowledge tree for each log. If the coverage rate falls below 90% , the LLM is prompted to generate additional easy questions targeting the missing leaf nodes. By enforcing high coverage, the process guarantees that relevant knowledge points are represented in the synthetic instruction set.

2) Diversity: The diversity dimension evaluates the variety of instruction questions generated for the same log. High diversity in instruction questions prevents overfitting and provides a broader range of learning scenarios for the follow-up training [32], [51]. Following previous works [32], we compute the ROUGE-L overlap between newly generated instructions and existing instructions for the same log. For different instruction questions on the same log, only instructions with a ROUGE-L overlap score less than 0.5 are retained, ensuring sufficient similarity between questions. If ROUGE-L exceeds 0.5, the LLM is prompted to rewrite the instructions. The rewriting process emphasizes semantic variation while maintaining logical consistency with the original instruction.

3) Correctness: The correctness evaluates the accuracy of the responses generated for each instruction question. The correctness of responses is critical for ensuring the reliability of the instruction dataset. Each instruction-response pair is verified by the LLM itself by re-referencing the knowledge tree, the verification determines whether the response aligns with the provided knowledge. If responses that deviate from the knowledge tree or exhibit inaccuracies, the LLM is prompted to regenerate corrected answers.

This evaluation and refinement stage is a cornerstone of the LogInstruct framework, ensuring that the synthesized instruction dataset is of high quality.

D. Finetuning the LLM with Instructions

After the creation of the instructions, we merge all datasets with different difficulty levels to fine-tune the open-source LLM. Following the standard instruction-tuning process, it takes one foundation LLM as the backbone, then the LLM gradually aligns its output with ideal log analysis results by predicting the next tokens in the *Response* conditioned on an input *Instruction*. Specifically, we concatenate the instruction and input log as a prompt and train the model to generate the instance output in a standard supervised way.

Denote the instruction instance by $\{(x_i, y_i)|x_i = instruction_i + input_i, y_i = output_i, i \in [1, N])\}$, which represents elements in the constructed instruction data set I. The optimization objective of SFT is to make the foundation LLM θ reasoning with professional analysis for each instruction and input log, which can be formulated as:

$$\mathcal{L}_{SFT} = argmax \sum_{i=1}^{N} logP(y_i|x_i; \theta) \quad (1)$$

TABLE I
STATISTICS OF DATASET USED FOR INSTRUCTION SYNTHESIS.

Vendor	Switches	Routers	Total
Huawei	4,415	3,614	8,029
H3C	2,208	2,924	5,132
Total	6,623	6,538	13,161

IV. EXPERIMENTS

A. Data Preparation

To mitigate the risk of data leakage from existing public system logs (e.g., distributed or operating system logs) that may have been included in LLM pretraining corpora, our experiments focus on network device logs, which are independently developed by vendors (e.g., Huawei, H3C) for specialized hardware and protocols, offering richer technical semantics. Due to rapid product iterations and customized development, vendor-specific logs are rarely included in publicly available LLM pre-training datasets.

We manually collect 13,161 unique logs and related documentation from the latest publicly available technical references provided by Huawei[1] and H3C[2] (Detailed statistics are provided in Table I). All raw logs and documentation are also publicly available in our repository for subsequent research, where each log entry is accompanied by a corresponding documentation page, providing an explanation of the log, descriptions of its parameters, potential causes, and recommended handling procedures. Based on this raw data, we synthesize 88K instruction data with GPT-4o for log analysis.

B. Implementation Details

1) Instruction Synthesis Settings: LogInstruct requires only a single LLM to perform knowledge tree construction, instruction synthesis, and instruction refinement. In the main experiments, we utilize GPT-4o as the LLM, processing 13,161 log-documentation pairs to synthesize instructions. We utilize HTTP requests to invoke the OpenAI APIs and interact with GPT-4o (*gpt-4o-2024-08-06*). To enhance the diversity of the instructions, the *temperature* is set to 0.8 during synthesis.

2) Instruction Tuning Setting: Following the paradigm shift towards generative large language models pioneered by the GPT series, the decoder-only architecture has emerged as the dominant and often more performant framework in contemporary NLP [52], [53]. In line with this trend, our instruction tuning experiments utilize open-source LLMs based on the decoder-only Transformer architecture.

We employ the LLaMA-Factory [54] to fine-tune the Qwen2.5-Instruct [55] series of models, including 1.5B and 7B versions. This allows us to compare the performance of LLMs with different sizes on the synthesized instruction datasets. For fair evaluation, we set the batch size to 32, the learning rate to 3.0e-5 with a cosine learning rate scheduler, and train for 5

[1]https://support.huawei.com/enterprise/en/index.html
[2]https://www.h3c.com/en/Support

TABLE II
DATASETS OF DOWNSTREAM TASKS LOG FOR EVALUATION

Tasks	Task Type	Huawei		H3C	
		Switches	Routers	Switches	Routers
Log Interpretation	Generation	3,885	3,656	1,821	2,566
Log Summarization	Generation	2,295	1,961	-	-
Solution Recommendation	Generation	3,843	3,855	2,210	2,929
Potential Failure Prediction	Classification	2,653	2,338	1,508	2,220
Root Cause Analysis	Classification	1,673	1,252	-	-

epochs in all experiments. Our fine-tuning process is conducted on a Linux server with a single NVIDIA A100 GPU.

3) Evaluation Setting: We leverage vLLM[3] to accelerate the inference process on open-source LLMs. To increase the stability of LLM's output, we set the temperature of LLMs to 0 during evaluation. For fairness in comparison, all LLMs follow the same *in-context learning* setting with 4 examples and provide the same prompt and examples during inference.

C. Downstream Tasks

Traditional log analysis tasks, such as log parsing and anomaly detection, have been extensively studied in the past decade, with many datasets sourced from LogHub [56]. However, as recently demonstrated in benchmark studies on LogHub, many methods [10], [15], [16], [18], [43] have already achieved perfect or near-perfect parsing accuracy on these datasets. Similarly, datasets for anomaly detection often feature relatively simple anomaly patterns [12], [38], allowing simple machine learning models (e.g., KNN) to achieve nearly perfect performance. This leaves limited room for further improvement and fails to sufficiently evaluate the capabilities of advanced models.

To address these limitations and comprehensively evaluate model performance, we construct five downstream tasks based on network device logs, spanning two dimensions: log understanding and log analysis applications. Specifically, log understanding tasks include two tasks and focus on semantic understanding capabilities, which serve as the foundational skills required for log analysis. And application tasks include three tasks grounded in real-world operational scenarios, which aim at evaluating the practical application capabilities of models in analyzing logs. Table II provides detailed statistics for each task dataset. All publicly available datasets are hosted in our GitHub repository to facilitate reproducibility and future research. In the following sections, we introduce the five tasks and their evaluation metrics.

1) Log Understanding Tasks:

- *Log Interpretation (LI):* The Log Interpretation task aims to convert raw log data into clear, human-readable insights that highlight the key issues indicated in the logs. This task evaluates a model's ability to semantically understand the critical information embedded in logs and generate coherent explanations.
  Dataset and Metric: Following KnowLog, we build datasets from public product documentation containing log entries paired with expert-written explanations. To ensure high-quality, redundant or unfocused explanations are manually filtered, retaining only concise, key-information-centric explanations. The final datasets employ raw logs as input and their verified explanations as ground truth, which is a generation task.
  We employ *BLEU* and *BERTScore* as the evaluation metrics to capture lexical overlap semantic similarities, where BLEU measures n-gram (n = 1 to 4) overlap between generated and reference texts, and BERTScore evaluates semantic similarity using contextual embeddings.
- *Log Summarization (LS):* The Log Summarization task aims to extract and condense the essential information from raw logs into concise summaries, evaluating the model's ability to identify essential information.
  Dataset and Metric: We build datasets from the collected network device logs, where the structure of each network device log contains a brief "summary" field (e.g., Brief: AS_PATH_EXCEED_MAXNUM) and a "log message" field. To prevent data leakage, we remove the summary field and use the log message as input, with the summary field serving as ground truth.
  We adopt *BLEU* and *ROUGE-1* as the evaluation metrics. BLEU emphasizes lexical precision, ROUGE-1 measures the recall of overlapping single words between generated texts and reference texts. These metrics are widely used in summarization tasks [57] to balance conciseness and informativeness.

2) Log Application Tasks:

- *Solution Recommendation (SR):* The Solution Recommendation task provides actionable solutions to address issues indicated in logs. It evaluates a model's ability to analyze logs comprehensively and recommend relevant commands or steps for troubleshooting, which is a critical requirement in real-world operational scenarios.
  Dataset and Metric: We collect logs paired with troubleshooting solutions from Huawei and H3C public forums as datasets. Specifically, user-submitted issues containing error logs are collected as input, and the highest-voted solutions are used as ground truth.
  We employ *BLEU* and *ROUGE-L* as the evaluation metrics, where BLEU emphasizes exact command/term matching, and ROUGE-L measures the longest common subsequence (LCS) between generated and reference texts, capturing structural coherence at the sentence level.
- *Potential Failure Prediction (PFP):* This task focuses on predicting whether a given log indicates a potential system or device failure. It is framed as a binary classification problem to distinguish failure-indicative logs and notification-indicative logs.
  Dataset and Metric: The datasets are collected from Huawei and H3C Reference Guides, which annotate logs with risk levels (e.g., Emergency, Informational). Logs indicating failure to the system are marked as True, while others are labeled as False.
  As the binary classification task, we report *Precision*, *Recall*, and *F1-score* on the True (anomaly) class as evaluation metrics.
- *Root Cause Analysis (RCA):* Root Cause Analysis identifies the underlying causes of anomalies based on log data,

[3]https://github.com/vllm-project/vllm

TABLE III
PERFORMANCE COMPARISON WITH DIFFERENT LLMS ON GENERATION TASKS

Models	Log Interpretation (BLEU / BERTScore)				LS (BLEU / ROUGE-1)		Solution Recommendation (BLEU / ROUGE-L)			
	Huawei		H3C		Huawei		Huawei		H3C	
	Switches	Routers	Switches	Routers	Switches	Routers	Switches	Routers	Switches	Routers
Flan-T5	21.24 / 70.34	18.43 / 67.29	11.80 / 63.06	12.10 / 63.57	12.85 / 11.73	10.37 / 8.94	5.39 / 6.83	4.79 / 5.81	9.02 / 5.27	7.68 / 4.84
ChatGPT	36.38 / 74.74	34.15 / 72.52	36.08 / 73.84	34.64 / 74.05	66.05 / 27.43	68.81 / 23.89	26.29 / 17.51	23.85 / 18.96	31.60 / 9.98	33.01 / 9.97
GPT-4o	34.47 / 73.67	34.01 / 72.60	35.55 / 73.15	34.02 / 73.44	**69.85** / 26.21	**70.16** / 25.58	27.97 / 16.86	23.87 / 20.07	35.73 / 10.17	35.50 / 10.32
Qwen2.5-1.5B	35.81 / 74.55	32.51 / 72.23	35.60 / 73.12	34.16 / 73.62	67.10 / 20.41	64.03 / 18.73	24.98 / 6.64	24.50 / 7.21	20.12 / 4.54	22.65 / 4.36
Qwen2.5-3B	36.48 / 71.96	33.34 / 67.77	34.57 / 71.94	33.64 / 72.35	66.21 / 25.88	68.21 / 24.21	23.32 / 12.24	21.05 / 5.70	18.65 / 3.61	21.30 / 3.52
Mistral-7B	28.33 / 70.61	27.10 / 68.84	24.41 / 68.98	22.02 / 69.44	64.79 / 18.72	64.47 / 16.22	22.33 / 16.09	19.42 / 17.99	25.69 / 9.68	25.50 / 9.82
Qwen2.5-7B	22.82 / 70.66	20.33 / 68.95	25.09 / 71.06	25.89 / 71.10	61.10 / 25.11	62.79 / 23.59	28.79 / 14.96	27.91 / 17.62	30.37 / 9.03	31.34 / 9.03
Llama3.1-8B	25.37 / 70.15	27.16 / 69.78	29.16 / 70.08	28.17 / 70.80	65.51 / 26.22	63.23 / 23.55	26.00 / 21.38	23.59 / 26.80	34.97 / 10.17	34.69 / 10.55
Qwen2.5-1.5B-FT	36.76 / 75.09	34.31 / 73.02	**36.88** / 73.43	34.80 / 74.14	67.61 / 21.46	68.70 / 19.31	31.27 / 14.91	29.63 / 17.40	33.95 / 9.63	36.64 / 9.24
Qwen2.5-7B-FT	23.46 / 70.07	21.31 / 67.99	27.13 / 70.74	26.85 / 70.91	63.02 / 26.93	63.03 / 25.40	31.41 / 15.06	29.84 / 17.45	34.25 / 9.76	36.39 / 9.41
LogInstruct (Qwen2.5-1.5B)	39.83 / 77.49	40.05 / 76.78	35.85 / **77.34**	**35.80** / **77.77**	66.90 / **27.72**	65.92 / **26.21**	31.27 / 14.91	34.10 / **21.83**	48.01 / 11.43	47.19 / **11.71**
LogInstruct (Qwen2.5-7B)	**39.85** / **77.56**	**40.54** / **76.89**	35.59 / 76.68	35.08 / 76.84	65.51 / 26.22	67.28 / 25.31	**41.11** / **17.74**	**36.84** / 18.06	**49.91** / **15.52**	**48.92** / 11.50

which is framed as a multi-class classification problem where the model must assign logs to one of several predefined error categories.

Dataset and Metric: The datasets are constructed by collecting user-submitted issues from Huawei's public forums. Only discussions analyzing the causes of errors are included. The final dataset consists of five error categories, such as communication or device failures.

As an unbalanced multiclass classification task and considering the importance of different classes, we report *Accuracy* and *Weighted-F1* as evaluation metrics.

D. Baselines

We evaluate LogInstruct against the following baseline:

1) Pre-trained Language Models: We include **BERT** [8], [58], **Biglog** [59], and **KnowLog** [17] as baselines, where BERT is the first and most widely used pre-trained model for log analysis, Biglog excels in capturing essential features with pre-training on logs, and KnowLog enhances its performance with knowledge-enhanced pre-training on log corpus. These pre-trained models are fine-tuned on specific tasks before prediction. For each task, we manually annotate 200 training samples for fine-tuning. However, since these models are not designed for text generation, we limit their evaluation to classification tasks. For generative log analysis tasks, we employ Flan-T5 (version: *flan-t5-large*) [60] as a baseline. It is a widely used sequence-to-sequence model that has undergone large-scale instruction tuning, endowing it with instruction-following and text-generation capabilities suitable for our evaluation.

2) General proprietary LLMs: We select two proprietary LLMs (**ChatGPT** (*version: gpt-3.5-turbo-0613*) and **GPT-4o** (*version: gpt-4o-2024-08-06*)) as baselines due to their amazing application potential in multiple domains, where we query them with OpenAI APIs.

3) General open-source LLMs: Considering practical deployment resources, we include several open-source LLMs (under 8B parameters) spanning diverse model architectures, including **Qwen2.5-Instruct-[1.5B, 3B, 7B]**[55], **Mistral-7B-Instruct**, and **Llama3.1-8B-Instruct**[61].

*4) Task-specific fine-tuned (**FT**) LLM:* Instruction tuning is a common approach to adapt open-source LLMs for domain-specific tasks [20], [31]. For this baseline, we manually annotate 200 training samples for each task and combine them into an instruction dataset for fine-tuning. For fair comparison, we fine-tune it on the same open-source LLM (Qwen2.5-Instruct).

E. Evaluations

We evaluate LogInstruct by answering the following research questions (RQs):

1) RQ1: How effective is LogInstruct compared with the current mainstream LLMs on downstream tasks?: We conduct extensive experiments to evaluate the performance of LogInstruct, comparing it with baselines across a range of log analysis tasks. As shown in Tables III–IV, LogInstruct achieves SOTA performance across all tasks except log Summarization, where proprietary LLMs exhibit marginally superior results. Notably, LogInstruct surpasses the best baseline by an average of 16.88% F1-score on the Potential Failure Prediction task. This significant improvement validates the effectiveness of our knowledge-driven instruction synthesis framework in enhancing LLMs' capacity for log analysis.

First, while pre-trained models (BERT, BigLog, and KnowLog) achieve reasonable performance on classification tasks, their performance plateaus when faced with tasks requiring deeper semantic understanding and reasoning. The results from FLAN-T5 indicate a noticeable performance gap when compared to current mainstream LLMs. This gap can be primarily attributed to its more limited knowledge base and comparatively weaker instruction-following proficiency. Second, proprietary LLMs exhibit remarkable semantic understanding across general domains. While they excel in tasks like Log Summarization, where semantic understanding plays a dominant role, they fall short in other knowledge-intensive analysis tasks when compared to LogInstruct. Notably, Qwen2.5-1.5B fine-tuned with LogInstruct consistently outperforms proprietary LLMs in most log analysis tasks. This suggests that domain-specific instruction tuning can compensate for smaller LLM sizes by aligning LLMs with log analysis tasks and domain knowledge. General open-source LLMs underperform due to their lack of log-specific knowledge, which confirms that our synthesized instructions

TABLE IV
PERFORMANCE COMPARISON WITH DIFFERENT LLMS ON CLASSIFICATION TASKS

Models	Potential Failure Prediction (Precision / Recall / F1)				RCA (Accuracy / Weighted-F1)	
	Huawei		H3C		Huawei	
	Switches	Routers	Switches	Routers	Switches	Routers
BERT	66.31 / 86.37 / 75.02	82.18 / 81.39 / 81.78	73.32 / 80.34 / 76.67	66.85 / 58.29 / 62.27	25.88 / 27.41	30.43 / 21.13
Biglog	69.31 / 85.59 / 76.59	81.55 / 92.44 / 86.65	70.23 / 90.62 / 79.13	60.24 / 70.28 / 64.88	26.95 / 26.60	31.38 / 18.29
KnowLog	71.40 / 86.25 / 78.12	82.86 / 90.24 / 86.40	70.86 / 90.22 / 79.37	64.15 / 76.65 / 69.84	36.22 / 40.28	38.41 / 33.36
ChatGPT	72.36 / 80.15 / 76.06	84.78 / 80.43 / 82.55	73.50 / 74.39 / 73.94	67.18 / 75.84 / 71.25	9.80 / 8.92	16.21 / 13.59
GPT-4o	80.70 / 79.95 / 80.32	84.58 / 79.81 / 82.13	76.25 / 82.86 / 79.42	68.33 / 82.68 / 74.82	35.92 / 45.69	26.83 / 30.79
Qwen2.5-1.5B	71.31 / 80.39 / 75.58	83.82 / 77.73 / 80.66	73.01 / 73.38 / 73.20	64.17 / 73.83 / 68.66	17.45 / 25.83	8.22 / 9.35
Qwen2.5-3B	69.71 / 65.21 / 67.38	83.88 / 69.84 / 76.22	70.55 / 55.54 / 62.15	67.71 / 62.64 / 65.07	21.33 / 21.86	28.83 / 26.14
Mistral-7B	68.23 / 88.34 / 76.99	80.49 / 88.16 / 84.15	71.01 / 79.53 / 75.03	64.76 / 83.17 / 72.82	30.81 / 41.31	22.12 / 24.69
Qwen2.5-7B	73.24 / 75.91 / 74.55	86.22 / 74.06 / 79.68	75.15 / 72.88 / 74.00	67.20 / 73.42 / 70.18	25.46 / 27.69	29.79 / 33.83
Llama3.1-8B	73.61 / 73.04 / 73.32	86.40 / 73.11 / 79.20	74.52 / 67.54 / 70.86	62.16 / 70.77 / 66.18	17.63 / 22.14	20.44 / 23.64
Qwen2.5-1.5B-FT	70.52 / 80.09 / 75.00	83.13 / 76.15 / 79.49	72.88 / 73.68 / 73.28	63.45 / 74.23 / 68.42	16.55 / 24.10	7.82 / 7.85
Qwen2.5-7B-FT	72.98 / 76.38 / 74.64	85.97 / 74.97 / 80.09	74.54 / 70.56 / 72.50	66.76 / 73.42 / 69.93	35.98 / 44.40	30.59 / 34.87
LogInstruct (Qwen2.5-1.5B)	70.69 / 93.42 / 80.48	82.41 / 93.23 / 87.49	72.63 / 95.26 / 82.42	65.24 / 94.76 / 77.28	24.86 / 27.29	26.11 / 23.05
LogInstruct (Qwen2.5-7B)	**99.87 / 97.54 / 98.69**	**99.88 / 97.80 / 98.83**	**94.95 / 96.77 / 95.85**	**93.80 / 97.50 / 95.61**	**41.78 / 48.11**	**40.89 / 35.36**

effectively bridge the domain knowledge gap in open-source LLMs. Third, the task-specific fine-tuned (FT) LLM exhibits limited improvements, primarily due to the scarcity and simplicity of annotated data. In contrast, LogInstruct leverages domain knowledge and hierarchical difficulty modeling, enabling significant performance improvements across all tasks. This highlights the critical role of incorporating domain knowledge and varying cognitive complexity in developing LLMs capable of handling complex log analysis scenarios. Finally, Qwen2.5-7B tuned with LogInstruct consistently outperforms its 1.5B counterpart, indicating that stronger base models better utilize synthesized instructions. This suggests that selecting open-source LLMs with robust foundational capabilities can further amplify the benefits of instruction tuning.

To further evaluate the quality of responses generated by models fine-tuned with LogInstruct, we conduct a comparative analysis of their outputs. Specifically, we focus on the Solution Recommendation task for this analysis, as it demands longer, more coherent, and highly readable responses. Considering evaluation costs, we randomly sample 200 queries from the Huawei dataset for evaluation.

For each query, we collect responses from both the base LLM (before fine-tuning) and its counterpart fine-tuned with LogInstruct. An LLM-as-a-Judge approach [50] is then employed to compare these paired responses. The judge model is instructed to select the better response based on a comprehensive evaluation across three dimensions: Usefulness, Relevance, and Accuracy. To mitigate potential evaluation bias, we report results using two distinct judge models: GPT-4 and DeepSeek-R1. Considering that LLMs may be sensitive to the order of presented options, we follow the calibration method introduced by Wang et al. [62]. For each test case, we query the judge model twice: once with the LogInstruct response first, and once with the baseline response first. The final comparative result is aggregated from these two prompts.

The experimental results are shown in Fig. 3, which shows the win rate of the LogInstruct-tuned models against their base counterparts. The fine-tuned models demonstrate significantly higher response quality. The 1.5B model fine-tuned with LogInstruct achieves a win rate exceeding 45% . This performance gain becomes more pronounced with a stronger base model: the fine-tuned 7B model attains a win rate of over 65% . These results indicate that our instruction tuning effectively enhances the models' ability to follow complex instructions and produce superior responses.

In conclusion, the performance gains highlight the effectiveness of LogInstruct in synthesizing high-quality instruction data for log analysis tasks. By incorporating systematic domain knowledge and hierarchical difficulty modeling, LogInstruct empowers LLMs to tackle a wide range of log analysis tasks with enhanced reasoning capabilities.

2) RQ2: How effective is LogInstruct in generalization ability?: To evaluate the generalization ability of the instructions synthesized by logInstruct, we design experiments with a focus on its performance in analyzing unseen logs. While the synthetic instructions are derived from logs of Huawei and H3C, including two devices: Switches and Routers, we additionally collect logs from Security devices, which are not part of the instruction synthesis process. This setup ensures that our evaluation measures the model's ability to generalize in an unseen domain. For the evaluation of LogInstruct's practicality, we focus on the log application tasks, as they reflect real-world operational scenarios. Specifically, we evaluate three log application tasks and one log understanding task using Security device logs. Of these tasks, the RCA task is evaluated using Huawei Security device logs only, as data from other vendors was unavailable. For the remaining tasks, we report the average results across the HW and H3C Security datasets.

The results of the experiments are shown in Fig. 4, which indicates that across all tasks, LLMs fine-tuned on our synthesized instruction data consistently achieved the best performance, significantly outperforming the baselines. Notably, the improvement on the Potential Failure Prediction task is particularly pronounced, where the F1-score of LogInstruct exceeded the best-performing baseline, GPT-4o, by an impressive 23.36% . First, when compared to the general LLMs, LogInstruct exhibits substantial improvements, which demonstrates that the knowledge-driven instructions endowed the models with domain-specific insights, enabling them to generalize effectively to diverse scenarios. Second, task-specific fine-tuned

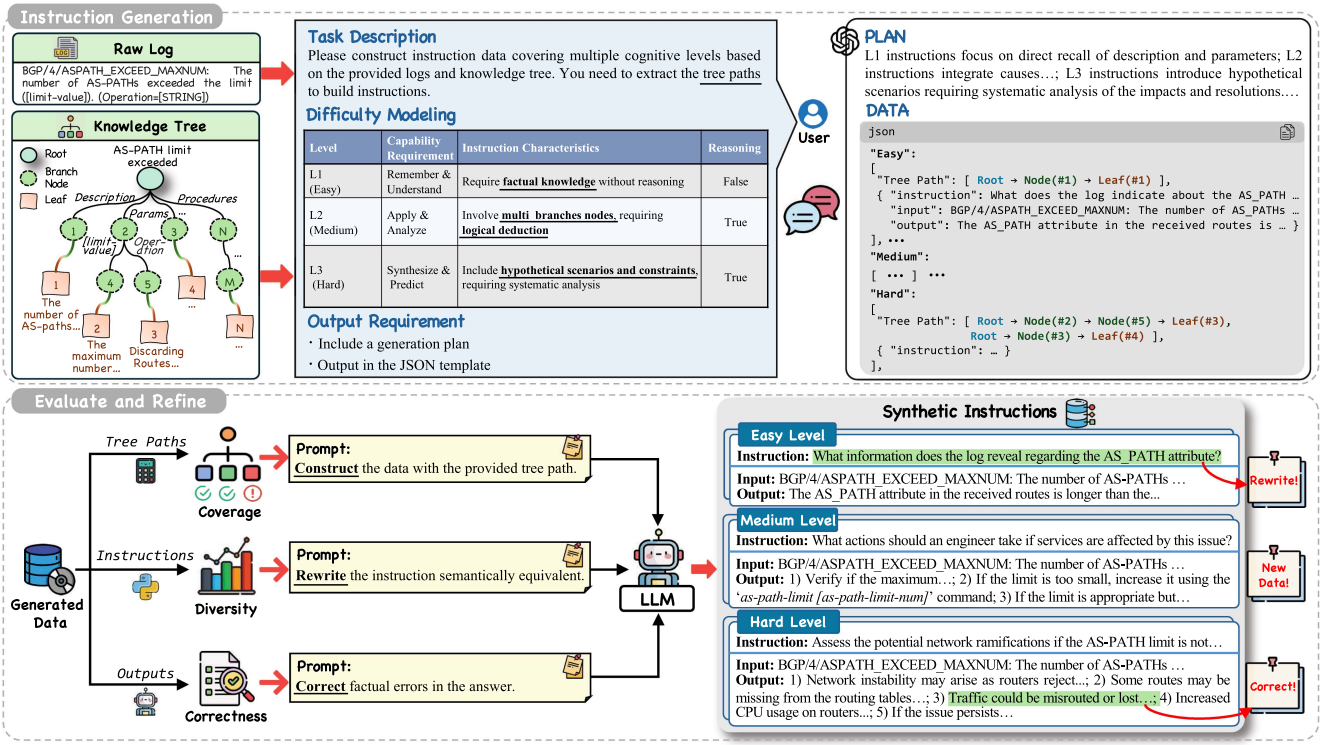

Fig. 2. The framework of Instruction Synthesis, which comprises two components: *LLM-based Instruction Generation* and *Evaluation and Refinement*. The first component synthesizes instruction data of different difficulty levels with the knowledge tree. The last component evaluates and refines the synthesized instructions to ensure the quality.

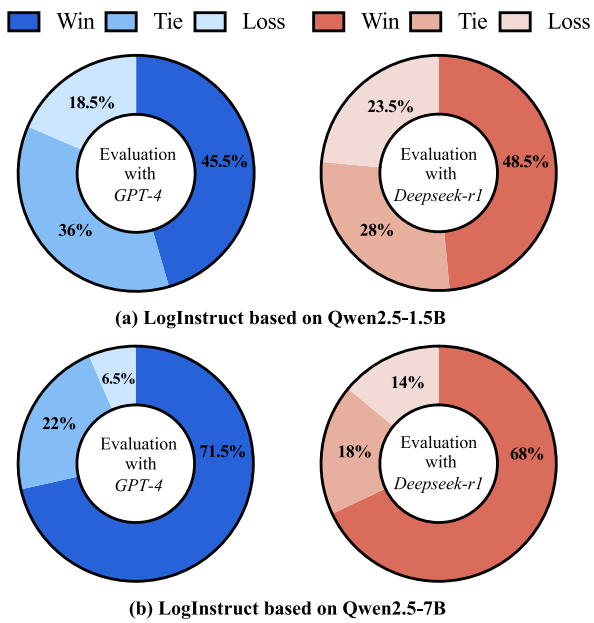

Fig. 3. Evaluation results of LogInstruct on Solution Recommendation task.

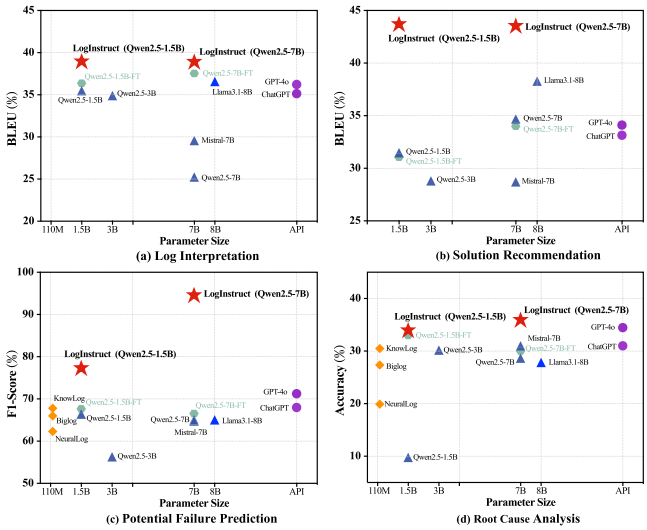

Fig. 4. Performance of LogInstruct on unseen logs.

LLMs (FT-LLMs) show limited improvements due to the lack of diverse data, and LogInstruct shows consistently better performance. The diverse and hierarchical instructions synthesized by LogInstruct equipped LLMs to handle diverse domain logs more effectively, alleviating inherent gaps in understanding logs and tasks.

A critical requirement for log analysis models in production is robust generalization, as real-world system logs are inherently unstable. Logging statements are frequently updated by developers during software evolution, leading to the continuous emergence of new yet semantically similar log messages.

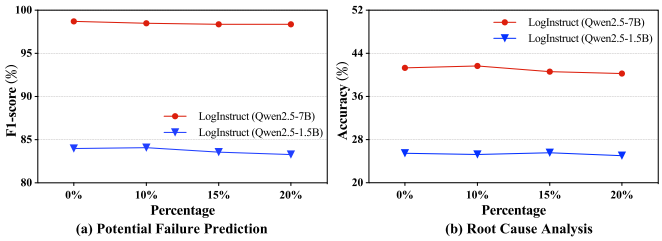

Fig. 5. Performance of LogInstruct on evolved logs.

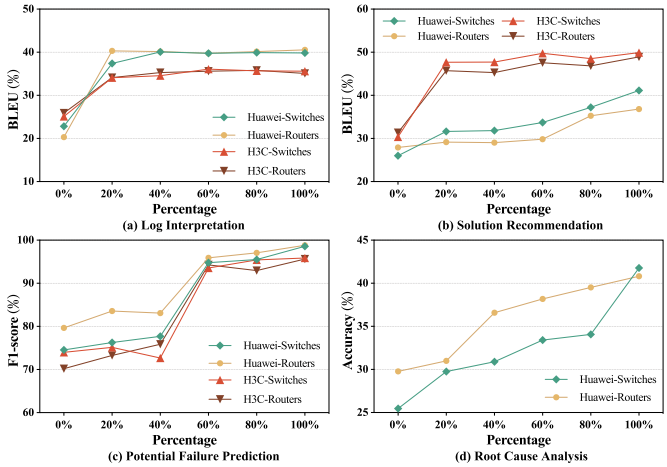

Fig. 6. Performance of LogInstruct with different proportions of knowledge injection.

Studies indicate that approximately 20% –45% of logging statements may change throughout a system's lifetime [6]. To assess whether models fine-tuned with LogInstruct merely memorize static log patterns or learn to perform generalized reasoning, we evaluate their performance on evolved logs. We conduct this evaluation on two tasks: Potential Failure Prediction and Root Cause Analysis. Following prior works [6], [63], we simulate log evolution by applying semantics-preserving transformations to the original Huawei log data. Specifically, we perform synonym replacement on the original log messages using WordNet, substituting certain tokens with their synonyms while strictly preserving the original semantic meaning and associated labels. To test varying degrees of change, we apply different replacement ratios based on the token count of the original logs.

The results are presented in Fig. 5, which demonstrates that models fine-tuned with LogInstruct maintain stable performance on the evolved logs across both tasks, with no noticeable degradation. This consistency indicates that the models' effectiveness does not rely on memorizing specific log instances. Instead, it suggests that the synthetic instruction data enhances the model's ability to generalize understanding, enabling it to perform precise analysis. This provides strong evidence for the effectiveness of this framework in boosting model generalization capabilities.

Consequently, the results affirm that LogInstruct exhibits strong generalization capabilities, synthesizing transferable domain knowledge and diverse instructions. This adaptability makes LogInstruct a promising tool for tackling diverse real-world log analysis scenarios.

3) RQ3: How effective is LogInstruct with different proportions of knowledge injection?: To investigate the impact of knowledge coverage density on the effectiveness of synthetic instructions, we conduct experiments by sampling instruction data for the same log entry at varying proportions and training Qwen2.5-7B on these subsets. This allows us to control the proportion of domain knowledge accessible for each log entry. Given that the log summarization task primarily focuses on semantic understanding with minimal reliance on knowledge, we exclude it from the evaluation and focus on the remaining four tasks.

As shown in Fig. 6, the results reveal several insights regarding the relationship between knowledge injection proportions and model performance. First, as the proportion of injected domain knowledge increases, the model's performance improves consistently across all tasks. This underscores the importance of domain knowledge in synthetic instruction generation, especially for complex log analysis tasks. Second, different log analysis tasks exhibit varying levels of dependency on domain knowledge. For relatively simpler tasks such as Log Interpretation, injecting 20% of the domain knowledge already leads to a significant performance boost. Beyond this threshold, the performance plateaus. In contrast, more complex tasks such as Root Cause Analysis, which involve reasoning and multi-step inference, show continuous performance improvements as the proportion of injected knowledge increases. Third, domain knowledge must reach a critical mass before the model exhibits a breakthrough in performance. For example, in the Potential Failure Prediction task, the model demonstrates slow performance gains when the knowledge proportion is below 40% . However, when the proportion increases to 60% , the model's performance experiences a breakthrough, with an F1 score improvement of 17.1% on average. This suggests that insufficient domain knowledge fails to unlock the model's full potential, while a more comprehensive knowledge coverage enables the model to achieve significant capability gains.

In conclusion, domain knowledge is essential for boosting model performance, particularly for complex reasoning tasks. While simpler tasks benefit from minimal knowledge injection, more complex tasks require a critical mass of domain knowledge to unlock the model's full potential.

4) RQ4: How effective is LogInstruct at different difficulty levels of instruction?: To investigate how instruction difficulty impacts the fine-tuning of log analysis LLMs, we systematically evaluate the LLM trained on instruction sets of varying complexity: Easy, Medium, Hard, and their combinations. The evaluation is performed on four downstream tasks, focusing on the log application task given its practicality.

The results are shown in Fig. 7, which reveals that different difficulty levels of instructions play complementary roles in enhancing log analysis capabilities. First, we find that easy instructions yield the most significant improvement and outperform those fine-tuned on medium or hard instructions, unlike prior findings in mathematical reasoning and code generation [64], [65], [66], [67], where complex instructions are critical for

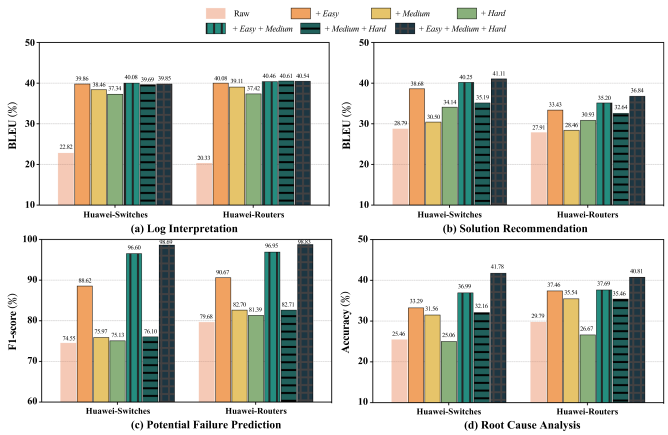

Fig. 7. Performance of different difficulty levels of instruction.

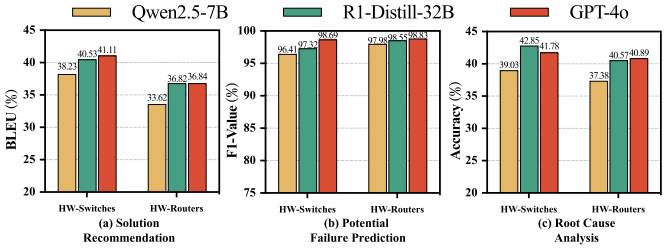

Fig. 8. Performance of different open-source LLMs for instruction Synthesis with LogInstruct framework.

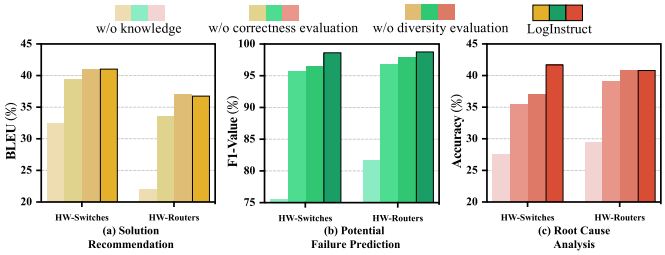

Fig. 9. Ablation studies for LogInstruct.

enhancing reasoning. Especially in the RCA task, the model fine-tuned only on hard instructions exhibits a performance drop. Even hybrid medium or hard instruction sets underperform easy instructions alone, highlighting the foundational role of basic domain knowledge in bridging the inherent knowledge gap of general LLMs. Hence, we can infer that easy instructions explicitly incorporate domain knowledge, which serves as a prerequisite for advanced reasoning. Without this grounding, complex instructions become ambiguous, leading to hallucinated or incoherent responses. Second, while easy instructions provide the foundational knowledge, combining them with medium and hard instructions further enhances the model's reasoning capabilities. Easy instructions establish the knowledge infrastructure, while complex instructions utilize and connect this knowledge for advanced inference tasks.

In summary, easy instructions are indispensable for addressing the domain knowledge gap in general LLMs, serving as prerequisites for complex log analysis capabilities. Furthermore, combining easy and complex instructions ensures balanced improvements in foundational knowledge and reasoning capabilities, highlighting the importance of instruction diversity and progression in fine-tuning LLMs for log analysis. LogInstruct ensures that the model is both knowledgeable and capable of handling complex reasoning tasks.

5) RQ5: How effective is LogInstruct based on open-source LLMs for instruction synthesis?: To address enterprise concerns around data privacy and proprietary LLMs dependencies, we evaluate whether LogInstruct can synthesize high-quality instructions using open-source LLMs as substitutes. Specifically, we evaluate two open-source LLMs of varying capabilities (Qwen2.5-7B and DeepSeek-R1-Distill-Qwen-32B) to analyze the impact of LLM's capacity on instruction synthesis quality. For the evaluation, we fine-tune the Qwen2.5-7B model and report results on three application tasks due to their comprehensive requirements for both semantic understanding and reasoning capabilities.

As shown in Fig. 8, models fine-tuned on instructions synthesized by R1-Distill-32B achieve comparable results with GPT-4o synthesized data, demonstrating that open-source LLMs can effectively replace proprietary models for instruction synthesis, addressing concerns around data privacy and API costs. On the other hand, Qwen2.5-7B synthesized instructions underperformed the other two models. The performance gap between Qwen2.5-7B and R1-Distill-Qwen-32B highlights the importance of choosing a base model with sufficient semantic understanding and reasoning capabilities for instruction synthesis.

In conclusion, LogInstruct can effectively utilize open-source LLMs to synthesize instructions for log analysis tasks, which highlights the practicality of open-source LLMs as replacements for proprietary LLMs, addressing privacy concerns and reducing API costs without compromising performance.

F. Ablation Studies

To further validate the importance of domain knowledge and instruction quality during instruction synthesis, we analyze the effects of removing key components in the instruction synthesis process and evaluate their impact on Qwen2.5-7B's performance. After fine-tuning Qwen2.5-7B with different instructions, we report results on three log application tasks.

1) Domain Knowledge Ablation: We remove domain knowledge from the instruction synthesis process, and the LLM is only provided with raw log data and tasked with designing multiple instructions directly. As shown in Fig. 9, results show that removing domain knowledge leads to a significant performance drop across all tasks. For example, on the Potential Failure Prediction task, the F1 score dropped by an average of 20.13% . This highlights the indispensable role of domain knowledge in guiding the instruction generation process. Without domain knowledge, the synthetic instructions often contain hallucinated responses and fail to address critical aspects of log analysis.

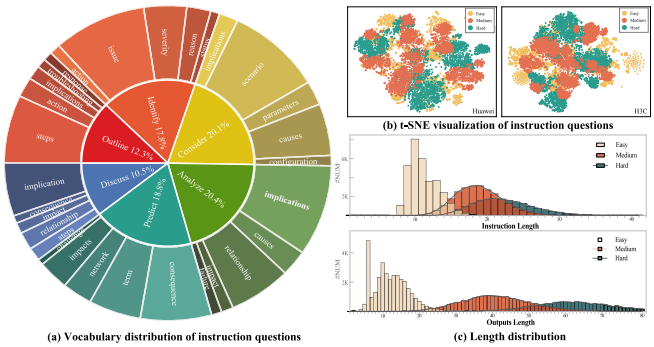

Fig. 10. Statistical analysis of instruction data.

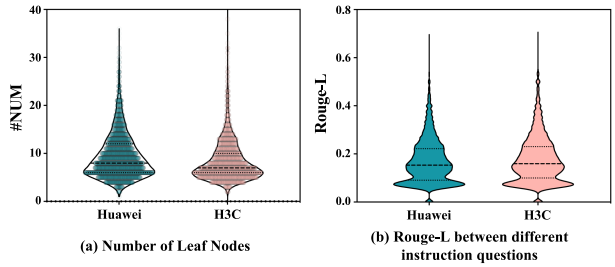

Fig. 11. Distribution of Leaf Nodes and Rouge-L among instruction questions.

2) Instruction Quality Ablation: Since the effect of knowledge coverage was already studied in RQ3, here we focus on the diversity and correctness of instructions and remove the corresponding evaluation modules in LogInstruct. It can be found that the removal of specific modules consistently leads to performance degradation, which highlights the importance of each evaluation module. However, the degree of performance decline varies, where the removal of correctness validation leads to a more pronounced performance decline compared to the removal of diversity validation. This result highlights that while diverse instructions contribute to broader task coverage, correctness is fundamental for ensuring reliable and accurate model outputs.

V. Discussion

A. Cost Analysis

To analyze the cost of synthesis instructions with LLMs, we count the API cost of GPT-4o. The average cost of constructing multiple instruction data for one log entry using GPT-4o is $**0.1**, with the cost of synthesizing one single instruction data averaging $**0.013**. This demonstrates that LogInstruct is both cost-efficient and scalable, making it a feasible solution for large-scale instruction generation.

B. Instruction Diversity

To quantitatively characterize the synthesized dataset, we first conduct an empirical analysis focusing on two aspects: the granularity of the constructed knowledge trees and the diversity of the generated instructions. First, we calculate the number of leaf nodes for each log entry in both the Huawei and H3C datasets, which reflects the amount of atomic knowledge extracted per log. The distribution is presented in Fig. 11(a). The result reveals that most logs in the Huawei and H3C datasets are associated with approximately 8 leaf nodes, demonstrating rich knowledge granularity. Second, we compute the ROUGE-L score between instruction questions generated for the same log. The distribution of these scores is shown in Fig. 11(b). The average ROUGE-L score is below 0.2, indicating low textual overlap among instructions derived from the same knowledge source.

To further evaluate the overall diversity of synthesized instructions, we first plot the distribution of semantic embeddings for all instruction questions (in Fig. 10(b)) and analyze their verb-noun structure using the Berkeley Neural Parser [68]. Specifically, we extract the root verbs and their first direct noun objects, and plot the top 6 most common root verbs and their top 5 direct noun objects in Fig. 10(a). The results demonstrate diverse intents and textual formats in the instructions. In Fig. 10(c), we also present the length distributions of the instruction questions and output responses, where we observe a positive correlation between instruction difficulty and the length of questions and responses. These findings underscore the ability of LogInstruct to synthesize high-quality, diverse instruction data. In addition, we manually verified 200 cases from the synthesized dataset and found no significant errors.

C. Case Studies

1) Instructions with Different Difficulty Levels: To illustrate the effectiveness of instruction synthesis with varying difficulty levels, we present a case study analyzing three log analysis instructions of increasing complexity: easy, medium, and hard.

As shown in Fig. 12, the easy instruction focuses on extracting parameters from the log (e.g., site ID, IP address, route distinguisher). This foundational task provides explicit domain knowledge, bridging the domain knowledge gap of general-purpose LLMs and enabling them to understand log syntax and semantics. The medium instruction requires reasoning about the potential consequences of ignoring the log alert, such as service disruptions or network instability. This level of complexity connects foundational knowledge to domain-specific reasoning, training the model to analyze operational impacts. The hard instruction demands a comprehensive action plan to mitigate service interruptions (e.g., scheduling changes, implementing redundancy, and monitoring traffic). It emphasizes strategic planning and multi-step reasoning, preparing the model for complex troubleshooting scenarios.

These examples highlight how hierarchical instructions progressively enhance LLMs' log analysis capabilities. Easy instructions inject essential knowledge, medium and hard instructions enable advanced reasoning capabilities. By synthesizing instructions across these difficulty levels, we ensure the progressive enhancement of LLM capabilities, enabling them to address diverse and complex log analysis tasks effectively. This

Fig. 12. Examples of instructions with different difficulty levels.

hierarchical framework not only improves task performance but also highlights the value of combining foundational knowledge with advanced reasoning in developing robust log analysis LLMs.

2) Instructions about Whether Introducing Knowledge: To illustrate the importance of domain knowledge in synthesizing effective log analysis instructions, we provide a comparative case study involving three examples of instruction synthesis: one generated by our LogInstruct framework, which integrates domain knowledge, and two generated by GPT-4o without access to domain knowledge.

As shown in Fig. 13, the first example demonstrates the effectiveness of LogInstruct in synthesizing accurate and actionable instructions. By leveraging domain knowledge such as command usage and diagnostic procedures, the generated instruction aligns with real-world log analysis practices, enabling engineers to address the issue efficiently. The second example shows that the absence of domain-specific understanding leads to an instruction that deviates from the actual problem context. The instruction assumes that configuring the port as a backup will resolve the speed difference issue, but this solution is irrelevant to the actual problem. Such outputs can mislead engineers, resulting in wasted effort and delayed troubleshooting. The third example shows that the output contains hallucinated information. The log message does not indicate a complete failure of the stack port, nor does it mention communication loss. Such inaccuracies arise from the lack of domain knowledge and can lead to critical misunderstandings in log analysis tasks. This case study demonstrates that incorporating

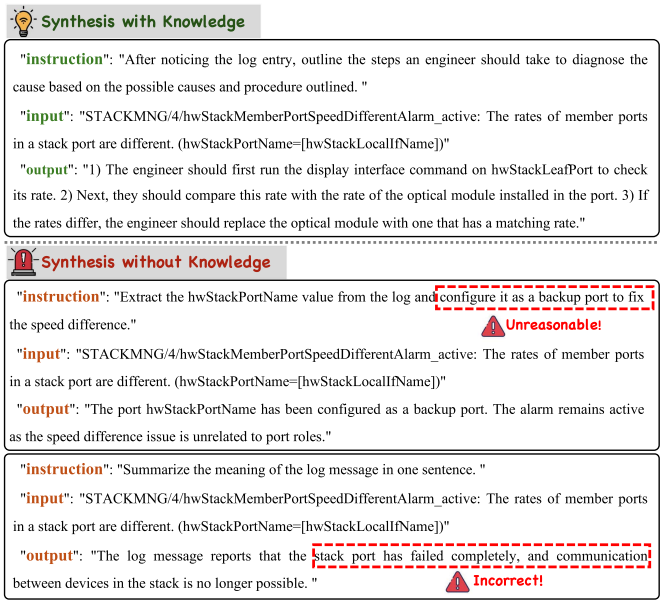

Fig. 13. Examples of instructions about whether introducing knowledge.

domain knowledge is indispensable for synthesizing high-quality instructions.

D. Threats to Vadility

1) Privacy Issue: Log data often contains sensitive information, raising privacy concerns when using proprietary LLMs. To address this, LogInstruct is designed as a general framework that supports integration with both proprietary and open-source LLMs, enabling local data processing and avoiding risks associated with external APIs.

2) Randomness: Randomness in LLM inference and prompt construction may pose a threat to the validity of our experiments. To ensure stability, we set the *temperature* to 0 for all LLMs during evaluation, ensuring stable and deterministic outputs. Furthermore, all LLMs are evaluated using the same prompt templates and examples. This consistency minimizes bias introduced by prompt variation and ensures fairness in comparative evaluations.

3) Data Leakage: Data leakage is also a potential threat to validity in studies employing LLMs to evaluate, wherein LLMs may have inadvertently memorized content from their extensive pre-training corpora. This could compromise the authenticity of their performance on evaluation data. In our work, we argue that this risk is minimal. The pre-training data for general-purpose LLMs primarily consists of web texts, news, books, and encyclopedias (e.g., Wikipedia) [52]. In contrast, the network device logs used in our study are vendor-specific and collected from recent software versions, making their appearance in general public corpora highly unlikely.

To empirically support this claim, we conduct a comparative analysis between 1,000 randomly sampled Wikipedia passages (representing a common pre-training corpus) and 1,000 log messages from our dataset. First, we compute the perplexity of a general LLM on both datasets. As shown in Fig. 14, the

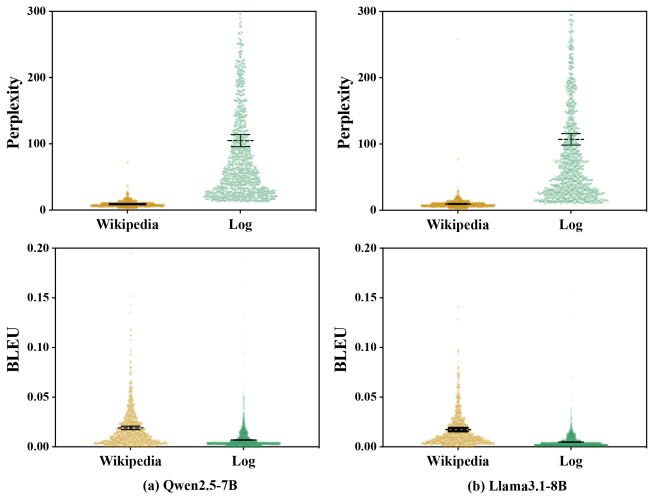

Fig. 14. Distribution of Perplexity and BLEU scores for LLMs on Wikipedia and Log corpus.

LLM exhibits significantly lower perplexity on Wikipedia text, indicating high familiarity. Conversely, the perplexity on log data is markedly higher, suggesting the LLM is far less exposed to such content. Second, we assess the LLM's ability to complete partial sequences. We provide the first 30% of tokens from each sample and prompt the LLM to complete the following content. We then calculate the BLEU score between the generated content and the original ground truth. As shown in Fig. 14, the average BLEU score is high for Wikipedia completions but negligible (below 0.01) for log completions, further confirming the LLM's lack of prior exposure to the specific log patterns.

These results indicate a very low risk of data contamination from pre-training, thereby strengthening the validity of our experimental findings.

E. Limitation

First, the current experiments in this work focus on network device logs. Although the results are encouraging, the applicability of LogInstruct to other domains remains unexplored. Real-world systems often generate diverse types of logs across different industries, such as cloud services, financial systems, and IoT environments. Future work will expand the evaluation to a broader range of log domains to validate the framework's adaptability and robustness in more diverse scenarios.

Second, LogInstruct employs the knowledge tree constructed from publicly available documentation to guide instruction synthesis. While this approach is effective in organizing domain knowledge, fault reports in real-world environments often incorporate diagnostic processes of experts, which are not currently utilized. Integrating such expert-driven diagnostic processes could significantly enhance the model's reasoning capabilities and better learn expert-level cognitive processes. In the future, we hope to collaborate with companies to obtain real failure reports for synthesizing richer, more reasoning-enhanced instructions.

Third, our study fine-tunes LLMs based on the prevalent decoder-only architecture. We have not explored the impact of our synthesized data on encoder-decoder architectures (e.g., T5-based models). A broader comparative study across different model architectures in the future could yield more generalizable insights into the framework's effectiveness.

VI. Conclusion

In this paper, we introduce LogInstruct, a knowledge-driven instruction synthesis framework designed to enhance LLMs' capabilities in automated log analysis. By incorporating domain knowledge through a carefully constructed knowledge tree and leveraging Bloom's Taxonomy to design hierarchical instructions, LogInstruct improves the capabilities of LLMs in log analysis. Through the synthesis of 88K high-quality instructions and fine-tuning of Qwen2.5 models, we achieve state-of-the-art performance across five log analysis tasks, demonstrating the critical role of domain knowledge and hierarchical instruction design in improving LLM performance.

References

[1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–37, 2021.
[2] S. Zhang et al., "Failure diagnosis in microservice systems: A comprehensive survey and analysis," *ACM Trans. Softw. Eng. Methodol.*, 2024.
[3] S. Zhang et al., "No more data silos: Unified microservice failure diagnosis with temporal knowledge graph," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 4013–4026, Nov./Dec. 2024.
[4] X. Zhang et al., "Onion: Identifying incident-indicating logs for cloud systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1253–1263.
[5] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
[6] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 807–817.
[7] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 92–103.
[8] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2021, pp. 492–504.
[9] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367.
[10] Y. Liu et al., "UniParser: A unified log parser for heterogeneous log data," in *Proc. ACM Web Conf.*2022, 2022, pp. 1893–1901.
[11] S. Yu, P. He, N. Chen, and Y. Wu, "Brain: Log parsing with bidirectional parallel tree," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3224–3237, Sep./Oct. 2023.
[12] B. Yu et al., "Deep learning or classical machine learning? An empirical study on log-based anomaly detection," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
[13] S. Zhang et al., "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 3851–3864, Jun. 2023.
[14] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable cross-system anomaly detection from logs with meta-learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–12.
[15] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "DivLog: Log parsing with prompt enhanced in-context learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–12.
[16] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 2438–2449.
[17] L. Ma et al., "KnowLog: Knowledge enhanced pre-trained language model for log understanding," in *Proc. 46th ieee/acm Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[18] Z. Jiang et al., "LiLac: Log parsing using LLMs with adaptive parsing cache," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 137–160, 2024.
[19] A. Zhong et al., "LogParser-LLM: Advancing efficient log parsing with large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 4559–4570.
[20] Y. Liu et al., "LogLM: From task-based to instruction-based automated log analysis," *IEEE/ACM 47th Int. Conf. Softw. Eng. Pract.*, pp. 401–412, 2025.
[21] J. Wang et al., "LogExpert: Log-based recommended resolutions generation using large language model," in *Proc. 2024 ACM/IEEE 44th Int. Conf. Softw. Eng.: New Ideas Emerg. Results*, 2024, pp. 42–46.
[22] L. Ma et al., "AdaptiveLog: An adaptive log analysis framework with the collaboration of large and small language model," *ACM Trans Softw. Eng. Methodol.*, pp. 1–43, 2025.
[23] M. Astekin, M. Hort, and L. Moonen, "A comparative study on large language models for log parsing," in *Proc. 18th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2024, pp. 234–244.
[24] J. Xu et al., "OpenRCA: Can large language models locate the root cause of software failures?," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 1–29.
[25] V.-H. Le, H. Zhang, and Y. Xiao, "Unleashing the true potential of semantic-based log parsing with pre-trained language models," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.*, 2025, pp. 711–711.
[26] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. web Serv.*, 2017, pp. 33–40.
[27] J. Zhu et al., "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2019, pp. 121–130.
[28] Y. Sui et al., "LogKG: Log failure diagnosis through knowledge graph," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3493–3507, May 2023.
[29] J. Achiam et al., "Gpt-4 technical report," 2023, *arXiv:2303.08774*.
[30] A. Liu et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.
[31] S. Zhang et al., "Instruction tuning for large language models: A survey," *ACM Comput. Surv.*, vol. 58, no. 7, pp. 1–36, 2026, .
[32] Y. Wang et al., "Self-instruct: Aligning language models with self-generated instructions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics* (Volume 1: Long Papers), 2023, pp. 13484–13 508.
[33] Z. Chen et al., "Agent-FLAN: Designing data and methods of effective agent tuning for large language models," in *Proc. Findings Assoc. Comput. Linguistics ACL 2024*, 2024, pp. 9354–9366.
[34] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "MagiCoder: Empowering code generation with OSS-instruct," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 52632–52 657.
[35] W. U. Ahmad et al., "OpenCodeInstruct: A large-scale instruction tuning dataset for code LLMs," 2025, *arXiv:2504.04030*.
[36] X. Zhang, C. Tian, X. Yang, L. Chen, Z. Li, and L. R. Petzold, "AlpaCare: Instruction-tuned large language models for medical application," 2023, *arXiv:2310.14558*.
[37] Z. Zhou et al., "LawGPT: A chinese legal knowledge-enhanced large language model," 2024, *arXiv:2406.04614*.
[38] N. Zhao et al., "An empirical investigation of practical log anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1404–1415.
[39] Z. Tan et al., "Large language models for data annotation and synthesis: A survey," in *Proc. 2024 Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 930–957.
[40] K. Wang et al., "A survey on data synthesis and augmentation for large language models," 2024, *arXiv:2410.12896*.
[41] M. T. Chandio, S. M. Pandhiani, and R. Iqbal, "Bloom's taxonomy: Improving assessment and teaching-learning process," *J. Educ. Educ. Develop.*, vol. 3, no. 2, pp. 203–221, 2016.
[42] S. Elkins, E. Kochmar, J. C. Cheung, and I. Serban, "How teachers can use large language models and Bloom's taxonomy to create educational quizzes," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 21, 2024, pp. 23084–230 91.
[43] Z. Ma, A. R. Chen, D. J. Kim, T.-H. Chen, and S. Wang, "LLMParser: An exploratory study on using large language models for log parsing," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
[44] Y. Liu et al., "Interpretable online log analysis using large language models with prompt strategies," in *Proc. 32nd IEEE/ACM Int. Conf. Prog. Comprehension*, 2024, pp. 35–46.
[45] C. Pei et al., "Self-evolutionary group-wise log parsing based on large language model," in *Proc. IEEE 35th Int. Symp. Softw. Rel. Eng.*, 2024, pp. 49–60.
[46] Y. Xiao, V.-H. Le, and H. Zhang, "Stronger, cheaper and demonstration-free log parsing with LLMs," 2024, *arXiv:2406.06156*.
[47] J. Qi et al., "LogGPT: Exploring ChatGPT for log-based anomaly detection," in *Proc. IEEE Int. Conf. High Perform. Comput. Commun., Data Sci. Syst., Smart City Dependability Sensor, Cloud Big Data Syst. Appl.*, 2023, pp. 273–280.
[48] W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "LogLLM: Log-based anomaly detection using large language models," 2024, *arXiv:2411.08561*.
[49] D. Zhang et al., "SciInstruct: A self-reflective instruction annotated dataset for training scientific language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 1443–1473.
[50] C. Xu et al., "WizardLM: Empowering large pre-trained language models to follow complex instructions," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 1–22.
[51] C. Zhou et al., "LIMA: Less is more for alignment," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 55006–550 21.
[52] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
[53] T. Wang et al., "What language model architecture and pretraining objective works best for zero-shot generalization?," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 22964–22984.
[54] Y. Zheng, R. Zhang, J. Zhang, Y. YeYanhan, and Z. Luo, "LLaMAfactory: Unified efficient fine-tuning of 100 language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 400–410.
[55] A. Yang et al., "Qwen2. 5 technical report," 2024, *arXiv:2412.15115*.
[56] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "LogHub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 355–366.
[57] H. Zhang, J. Cai, J. Xu, and J. Wang, "Pretraining-based natural language generation for text summarization," in *Proc. 23 rd Conf. Comput. Natural Lang. Learn.*, 2019, pp. 789–797.
[58] Y. Lee, J. Kim, and P. Kang, "LanoBERT: System log anomaly detection based on BERT masked language model," *Appl. Soft Comput.*, vol. 146, 2023, Art. no. 110689.
[59] S. Tao et al., "Biglog: Unsupervised large-scale pre-training for a unified log representation," in *Proc. IEEE/ACM 31st Int. Symp. Qual. Service*, 2023, pp. 1–11.
[60] H. W. Chung et al., "Scaling instruction-finetuned language models," *J. Mach. Learn. Res.*, vol. 25, no. 70, pp. 1–53, 2024.
[61] A. Grattafiori et al., "The LLaMA 3 herd of models," 2024, *arXiv:2407.21783*.
[62] X. Wang et al., "MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback," *Proc. 25th Int. Conf. Learn. Representations*, 2025, pp. 1–35.
[63] L. Ma et al., "Luk: Empowering log understanding with expert knowledge from large language models," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2764–2786, Oct. 2025.
[64] K. Zhou et al., "JiuZhang3. 0: Efficiently improving mathematical reasoning by training small data synthesis models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 1854–1889.
[65] Y. Huang et al., "Key-point-driven data synthesis with its enhancement on mathematical reasoning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 23, 2025, pp. 24176–24184.
[66] Z. Tang, X. Zhang, B. Wang, and F. Wei, "MathScale: Scaling instruction tuning for mathematical reasoning," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 47885–47 900.
[67] Z. Bi, N. Zhang, Y. Jiang, S. Deng, G. Zheng, and H. Chen, "When do program-of-thought works for reasoning?," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 16, 2024, pp. 17691–17 699.
[68] N. Kitaev and D. Klein, "Constituency parsing with a self-attentive encoder," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 2676–2686.

Lipeng Ma received the BS degree in information security from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2019. He is currently working toward the PhD degree in computer science with Fudan University. His research interests include knowledge graph applications, large language models, software engineering, and AIOps.

Yixuan Li received the master of Science (MSc) degree in applied mathematics from The Hong Kong Polytechnic University, in 2024. She is currently working toward the PhD degree with the College of Computer Science, Fudan University. Her research interests include code intelligence, large language models (LLMs), and point cloud completion (her MSc degree research focus).

Weidong Yang (Member, IEEE) received the PhD degree in software engineering from Xidian University, in 1999. From 1999 to 2001, he was a postdoctoral researcher with the School of Computer Science, Fudan University. He is currently a professor with the School of Computer Science, Fudan University, Shanghai, China. His research interests include Big Data, knowledge engineering, database and data mining, and software engineering.

Mingjie Zhou is currently working toward the PhD degree with Fudan University, Shanghai, China. His work has been authored or coauthored in data mining and software engineering international conferences. His research interests include AIOps and knowledge graph.

Mingyu Zhao received the PhD degree in computer science from Fudan University, China, in 2024. He is currently an assistant professor with Northwestern Polytechnical University, China. His research interests include data mining and machine learning.

Ben Fei (Member, IEEE) received the MS degree from the Department of Materials Science, Fudan University, Shanghai, China, in 2021, and the Ph.D. degree from the School of Computer Science, Fudan University, in 2024. He is currently a postdoc with the Chinese University of Hong Kong. His research interests include generative models, large language models, 3D computer vision, and AI for science. He is also an IEEE Young professional.

Shuhao Li (Graduate Student Member, IEEE) received the master's degree in computer technology from Guangzhou University, Guangzhou, China, in 2023. He is currently working toward the PhD degree with the School of Computer Science, Fudan University, Shanghai, China. His research interests include spatio-temporal data mining, anomaly detection, and large language models.

Sihang Jiang received the BSc degree in information security and the PhD degree in software engineering from Fudan University, in 2017 and 2024, respectively. He is currently a postdoctoral researcher with Fudan University. His research interests mainly include knowledge graph, large language models, and AIOps.

Yanghua Xiao received the PhD degree in software theory from Fudan University, Shanghai, China, in 2009. He is currently a professor of computer science with Fudan University, where he is also the director with Knowledge Works Laboratory. His research interests include Big Data management and mining, graph database, knowledge graph. He was a visiting professor with Human Genome Sequencing Center, Baylor College Medicine, and visiting researcher with Microsoft Research Asia.